Full Stack AI Engineer Assessment

Overview and objective

This sanitized generated fixture describes a Full Stack AI Engineer Assessment for DocuLens-style review tests. The candidate must build an AI-powered full-stack application that accepts a reviewer document, analyzes it with a large language model through a backend provider boundary, and lets the reviewer ask follow-up questions grounded in the selected source. The assessment values practical engineering judgment, a runnable end-to-end path, and clear trade-offs over architecture theater.

Application scope

The product should feel like a document review assistant rather than a raw prompt console. A reviewer signs in, adds a source document, sees a concise briefing, asks questions, and inspects evidence. The app may use a safe sample document, pasted text, or a text-based PDF upload. Scanned-image OCR is not required. The generated review should explain limitations honestly when the selected source does not support a claim.

Backend requirements

The backend must expose a REST API for authentication, document creation, analysis, chat, and source retrieval. It must include an endpoint that sends prompts to an LLM provider through a provider abstraction instead of calling a vendor directly from UI code. The backend must normalize model responses before returning them, persist documents and messages, and keep analysis results stable across save and reload boundaries. JWT authentication is expected for protected routes, and ownership checks must prevent one reviewer from reading another reviewer source.

AI and retrieval requirements

The AI path must combine document-level analysis with retrieval-grounded chat. Source text should be normalized, chunked, and searched before answering specific questions. The chat answer should cite evidence when retrieved chunks support the response, and it should use a clear insufficient-evidence state when the selected document does not

support a specific claim. Broad overview questions may produce a full-document overview with a caveat when precise citations are not available. Prompt construction, provider configuration, and response parsing should be explicit enough for review.

Frontend requirements

The frontend must be implemented in React and provide a user-friendly flow with source intake, review briefing, starter questions, chat input, answer cards, and inspectable evidence. It should include loading states, empty states, error states, and retry or refine actions. AI responses must be formatted as readable prose and structured sections, not raw JSON. The UI should help the reviewer re-ask or narrow a question, display uncertainty when appropriate, and keep the active source identity visible while switching between sources.

Data, privacy, and logging requirements

The application must describe what document data is stored, when it is sent to a third-party AI provider, and how long it is retained for the assessment demo. Logs must avoid raw document text, provider payloads, stack traces, local file paths, and secret-shaped values. PDF metadata shown to the reviewer should use a safe original basename, MIME type, size, source method, and upload time. Configuration and secure secret handling must keep provider keys, JWT secrets, database passwords, and cloud credentials out of source control and out of reviewer-facing output.

Reliability and evaluation requirements

The project should include targeted tests for authentication, ownership, ingestion, provider parsing, retrieval policy, chat answer states, and UI regression paths. Evaluation should include golden questions about the assessment document, including overview, backend, frontend, data and privacy, reliability, deployment, and deliverables. The implementation should fail safely when conversion, retrieval, provider, or network operations fail, preserving prior useful output and giving the reviewer a recovery action.

Deployment and operations requirements

The assessment expects deployment thinking with AWS infrastructure described or provisioned using Terraform or CloudFormation. The design should separate configuration from code, handle secrets through environment or managed secret stores, and explain scaling considerations for API, database, PDF processing, and AI provider latency. The demo infrastructure can be small and disposable, but the repository should make operational assumptions and teardown steps clear.

Deliverables

The candidate must deliver a Git repository with runnable local setup instructions and a README that explains architecture, AI design, data flow, privacy decisions, reliability strategy, deployment approach, and trade-offs. The README should include how to run the app, how to run targeted tests, how to configure the AI provider safely, and what is intentionally out of scope. Evidence of working behavior is more important than diagrams without implementation.

Review rubric markers

A strong submission demonstrates a coherent full-stack path, secure authentication, source-grounded AI answers, normalized structured output, useful frontend states, documented data handling, targeted automated tests, and credible AWS deployment choices. A weak submission exposes raw model payloads, invents unsupported facts, hides errors, omits persistence, ignores privacy, or cannot be run by the reviewer.